



MÓDULO 04: Control de Flujo, Bucles y Manejo de Excepciones

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora 30 minutos

Módulo: 04 - Flow Control, Loops & Exception Handling

1. Objetivos de Aprendizaje

- **Operadores Relacionales y Lógicos:** Dominar las comparaciones (`==` , `!=` , `>` , `<` , `>=` , `<=`) y las combinaciones lógicas (`and` , `or` , `not`).
- **Condicionales (`if` , `elif` , `else` , `match-case`):** Estructurar decisiones simples y compuestas en Python 3.10+.
- **Bucles Repetitivos (`for` , `while`):** Iterar sobre secuencias, controlar rangos con `range()` , usar las sentencias `break` , `continue` y `pass` .
- **Iteración Avanzada (`enumerate` , `zip`):** Recorrer secuencias en paralelo y gestionar índices implícitos de forma elegante.
- **Gestión de Excepciones (`try` , `except`):** Prevenir caídas del programa en tiempo de ejecución ante entradas numéricas o archivos corruptos.

SESIÓN 4: Control de Flujo y Bucles en Python

En este módulo aprenderemos a **controlar el orden de ejecución de las instrucciones** en un programa.

Hasta ahora, el código en Python que hemos escrito se ejecutaba **línea por línea, de arriba a abajo**.

Sin embargo, en la mayoría de los programas reales necesitamos tomar **decisiones** o **repetir acciones** varias veces.

Objetivos del módulo

- Comprender qué es el **control de flujo** en un programa.

- Aprender a usar las **estructuras condicionales**: `if`, `elif`, `else`.
 - Introducir el uso de **estructuras condicionales múltiples** con `match-case` (Python $\geq$ 3.10).
 - Conocer la construcción `try-except` para el **manejo de errores**.
 - Aprender a usar los **bucles** para repetir instrucciones:
 - `for` → recorrer colecciones o rangos de números.
 - `while` → repetir mientras se cumpla una condición.
 - Usar **palabras clave especiales** como `break` y `continue` para controlar los bucles.
-

Contenido

1. Operadores lógicos en Python
 2. Condicionales: `if`, `elif`, `else`
 3. Uso de `match-case` como alternativa a largas cadenas de `if-elif-else` (Python $\geq$ 3.10)
 4. El bucle `for`
 5. El bucle `while`
 6. Control de bucles con `break` y `continue`
 7. Manejo de errores con `try-except`
 8. Ejemplos prácticos y ejercicios
-

Idea clave:

El **control de flujo** permite que nuestros programas pasen de simples secuencias lineales a soluciones dinámicas capaces de **tomar decisiones** y **adaptarse a distintas situaciones**.

@Gustavo Liñán Cembrano

1. Operadores lógicos y relacionales en Python

En Python podemos usar **operadores relacionales** (comparaciones) y **operadores lógicos** (combinación de condiciones).

Ambos son fundamentales para tomar decisiones en los programas.

Tabla de operadores

Tipo	Operador	Descripción	Ejemplo	Resultado
Relacional	<code>==</code>	Igual a	<code>5 == 5</code>	True
Relacional	<code>!=</code>	Distinto de	<code>5 != 3</code>	True
Relacional	<code>></code>	Mayor que	<code>7 > 3</code>	True
Relacional	<code><</code>	Menor que	<code>2 < 8</code>	True
Relacional	<code>>=</code>	Mayor o igual que	<code>5 >= 5</code>	True
Relacional	<code><=</code>	Menor o igual que	<code>4 <= 2</code>	False
Lógico	<code>and</code> &	Devuelve True si todas las condiciones son verdaderas	<code>(5 > 2 and 3 < 4)</code>	True
Lógico	<code>or</code> \	Devuelve True si alguna condición es verdadera	<code>(5 < 2 or 3 < 4)</code>	True
Lógico	<code>not</code>	Invierte el valor lógico	<code>not(5 > 2)</code>	False

```
In [ ]: ## Ejemplos

a = 10
b = 5

# Operadores relacionales
print(a == b)  # False
print(a != b)  # True
print(a > b)   # True
print(a <= b)  # False

# Operadores lógicos
print(a > 0 and b > 0)  # True, ambas condiciones son ciertas
print(a < 0 or b > 0)   # True, la segunda condición es cierta
print(not(a > b))       # False, porque a > b es True y NOT lo invierte
```

2. Condicionales en Python: if, elif, else

En Python podemos usar estructuras condicionales para **ejecutar instrucciones solo cuando se cumpla una condición**.

Sintaxis básica: if ... else

La forma más sencilla de condicional es:

```
if condicion:
    # bloque de código si la condición es True
else:
    # bloque de código si la condición es False
```

```
In [ ]: edad = 20

if edad >= 18:
    print("Te puedes pedir una cerveza")
else:
    print("Para ti, mejor un Colacao fesquito")
```

Uso de `elif` en Python

Cuando tenemos **más de dos condiciones posibles**, no basta con `if ... else`. En esos casos usamos `elif` (*else if*), que nos permite encadenar varias condiciones de forma clara y ordenada.

Sintaxis de `elif`

```
if condicion1:
    # bloque de código si condicion1 es True
elif condicion2:
    # bloque de código si condicion1 es False y condicion2 es True
elif condicion3:
    # bloque de código si las anteriores son False y condicion3 es True
else:
    # bloque de código si ninguna condición es True
```

```
In [ ]: # Pedimos al usuario que introduzca una nota
nota_str = input("¿Qué nota le has puesto? ")

# Convertimos la cadena a número decimal
# Necesitamos float porque input() devuelve siempre un string
nota = float(nota_str)

# Evaluamos la nota usando condicionales
if nota >= 9:
    print("Sobresaliente")
elif nota >= 7:
    print("Notable")
elif nota >= 5:
    print("Aprobado")
else:
    print("Suspenso")

# Nota para el alumno:
# - Si introduces algo que no sea un número, el programa dará error.
# - Más adelante aprenderemos a controlar estos errores usando try-except.
```

Ejercicio: Crea una funcion que usando esos limites, pida al usuario la nota numerica e imprima si ha sacado sobresaliente/notable/aprobado/suspenso

```
In [ ]: #tu codigo aqui.
```

3. Uso de `match-case` en Python

En Python 3.10 y versiones posteriores, podemos usar `match-case` como alternativa a largas cadenas de `if-elif-else`.

`match-case` permite **comparar un valor con diferentes patrones** y ejecutar el bloque correspondiente.

No es algo que se use mucho, a mi criterio, es más habitual encontrar `if--elif--else`

Sintaxis básica

```
match variable:
    case valor1:
        # bloque de código si variable == valor1
    case valor2:
        # bloque de código si variable == valor2
    case _:
        # bloque de código si ningún valor coincide (equivalente a else)
```

```
In [ ]: #Ejemplo

dia = "sábado"

match dia:
    case "lunes":
        print("Hoy es el peor día de la semana")
    case "sábado" | "domingo":
        print("Estás de finde!!!")
    case _:
        print("Día entre semana")

#juega a cambiar el dia
```

Veamos como introducir más de una condición lógica para entrar en un bloque IF-ELSE

Recuerda:

- Los operadores relacionales-->> Para comparar
- Los operadores lógicos --> Para verificar condiciones lógicas (solo aceptan argumentos tipo True/False)

```
In [ ]: #Para entrar en la discoteca hay que ser mayor de 21 y no llevar calcetines

Juan_Edad = 20
Juan_Calcetines="blancos"

if( Juan_Edad >= 21 and Juan_Calcetines != "blancos"):
    print("Bienvenido al local")
else:
    print("Lo siento tenemos un evento privado")
```

Ejercicio guiado: Crea la funcion **portero()** de discoteca que pregunta por el nombre, la edad, y el color de los calcetines. Si la edad es de al menos 21 y los calcetines no son blancos, saludará por el nombre al usuario, y le dirá: bienvenido puedes entrar. Si no, le dirá que no puede entrar porque hay un evento privado.

```
In [ ]: #tu intento aqui!!!
```

```
In [ ]: ## Solución

def portero():
    """
    Función que simula la labor de un portero de discoteca.

    Pregunta al usuario por su nombre, edad y color de calcetines, y determina
    puede entrar al local según las siguientes reglas:

    - Debe tener al menos 21 años.
    - Sus calcetines no deben ser blancos.

    Mensajes mostrados:
    - Si cumple las condiciones: saluda al usuario por su nombre y dice que
    - Si no cumple las condiciones: indica que no puede entrar debido a un evento

    Uso:
        portero()

    No recibe parámetros y no devuelve ningún valor.
    """
    # Pedimos los datos al usuario
    nombre = input("¿Cuál es tu nombre? ")
    edad_str = input("¿Cuántos años tienes? ")
    calcetines = input("¿De qué color son tus calcetines? ").lower()
    # Convertimos a minúsculas para hacer más fácil la comparación

    # Comprobamos que la edad es un número
    if not edad_str.isdigit(): #Observa como chequeamos que la edad sea un número
```

```
print("Debes introducir un número válido para la edad")
return

edad = int(edad_str) #no nos interesan los decimales.

# Aplicamos las reglas
if edad >= 21 and calcetines != "blancos":
    print(f"Hola {nombre}, bienvenido, puedes entrar")
else:
    print(f"Lo siento {nombre}, no puedes entrar porque hay un evento pr

# Llamada a la función
portero()
```

3. El bucle `for` en Python

El bucle `for` en Python se utiliza para **recorrer elementos de una secuencia** (listas, cadenas de texto, rangos de números, diccionarios, etc.) y ejecutar un bloque de instrucciones repetidamente.

Sintaxis básica

```
for variable in secuencia:
    # bloque de instrucciones
```

variable toma en cada iteración el valor de un elemento de la secuencia.

El bucle se repite hasta recorrer todos los elementos, por orden

Instrucciones de control en los bucles: `break` y `continue`

En Python podemos alterar el flujo normal de un bucle usando estas dos instrucciones:

`break`

- Sirve para **interrumpir el bucle por completo** y salir de él inmediatamente.
- Se usa, por ejemplo, cuando ya hemos encontrado lo que buscábamos y no tiene sentido seguir iterando.

```
In [ ]: ##### Ejemplo:
secuencia = [10,11,12,5, 13,14]

for numero in secuencia:
    print(numero)
```

```
#### Ejemplo de destruccion masiva!!!
secuencia = [10,11,12,5, 13,14]
```

```
for numero in secuencia:
    secuencia.append(numero+1)
    print(numero)
    if(len(secuencia)>1000):
        break
"""
```

El bucle for en Python no copia la lista, sino que la recorre elemento a elemento.

-- En cada iteración mira el siguiente índice de la lista dinámicamente.

-- Esto significa que si la lista crece, el bucle seguirá recorriendo los números.

Como dentro del for se hace `secuencia.append(numero+1)`, la lista crece indefinidamente sin el if..... seguiría hasta colapsar la memoria del equipo ya que la variable seguiría creciendo de tamaño constantemente

La función `range()` genera secuencias de números enteros, muy útil para controlar bucles.

```
In [ ]: # range(inicio, fin, paso)
# inicio: primer número (por defecto 0)
# fin: se detiene en fin-1
# paso: incremento (por defecto 1)

for i in range(5):
    print(i) # 0, 1, 2, 3, 4 Recuerda, siempre el primer elemento está en 0
print('-----')
```

```
Valores= range(10)
iteracion = 1
print(type(Valores))
for elemento in Valores:
    print(f'valor: {elemento} Iteracion: {iteracion}')
    iteracion += 1
    #Aprendemos este modo de incrementar/decrementar una variable
    # a += 1 es lo mismo que a = a+1

print('-----')
```

```
for i in range(2, 10, 2):
    print(i) # 2, 4, 6, 8
```

```
In [ ]: for numero in range(5):
    if numero == 2:
        continue # saltamos la iteración cuando numero == 2
    print(numero)
```

continue

Sirve para saltar a la siguiente iteración sin ejecutar el resto del código dentro del bucle en esa vuelta.

Se usa, por ejemplo, para ignorar ciertos valores pero continuar con el bucle.

A veces queremos detener el bucle antes de tiempo. Para ello se usa `break`. Por ejemplo lo ejecutamos mientras una variable, cuya evolución desconocemos, cumpla alguna condición. Imagina que tienes un csv de lecturas de temperatura que van creciendo, junto a otras lecturas de, pongamos radiación UV, y quieres ir reportando la radiación UV mientras la temperatura fue menor de 40°C. Para ello tenemos `break`.

```
In [ ]: for i in range(10):
        print(i)

        if i == 5:
            print("Me paro en 5")
            break
```

Además de `break`, es habitual usar `pass` cuando no queremos hacer nada, o todavía no tenemos una parte de código terminada pero no queremos errores. Por ejemplo

```
def funcion_por_definir():
    pass    # Implementaré esto después
```

```
In [ ]: for i in range(10):
        if i!=5:
            print(i)
        else:
            pass    #Aun no se lo que haré si i es 5.....ya veré
```

A veces, nos interesa recorrer la secuencia claves, valores de un diccionario.

```
In [ ]: edades = {"Ana": 25, "Luis": 30, "Marta": 22}

# Solo claves
for clave in edades:
    print(clave)
print("-----")
# Claves y valores
print("Imprimimos claves y valores\n\n\n") #Usamos \n para añadir un salto de línea
for nombre, edad in edades.items():
    print(f"{nombre} tiene {edad} años")
```

Si queremos recorrer en un mismo bucle varias secuencias tenemos la sentencia `zip()`

Uso de `zip()` en bucles `for`

La función `zip()` nos permite **recorrer varias secuencias a la vez**.

Combina elementos de varias listas (o secuencias) en tuplas, emparejando los elementos que ocupan la misma posición.

Sintaxis

```
for a, b, c, ... in zip(seq1, seq2, seq3, ...):  
    # bloque de instrucciones
```

Cada iteración devuelve una **tupla** con los elementos de las secuencias en la misma posición.

IMPORTANTE El bucle se detiene cuando se alcanza el final de la secuencia más corta.

```
In [ ]: nombres = ["Ana", "Luis", "Marta"]  
edades = [25, 30, 22, 35]  
ciudades = ["Madrid", "Sevilla", "Valencia", "Caceres", "Barcelona"]  
for nombre, edad, ciudad in zip(nombres, edades, ciudades):  
    print(f"{nombre} tiene {edad} años y vive en {ciudad}")
```

`enumerate()` en Python

La función `enumerate()` nos permite recorrer una secuencia (lista, tupla, cadena, etc.) obteniendo en cada iteración **el índice y el valor** del elemento.

Es muy útil cuando necesitamos saber **en qué posición** del bucle estamos.

Sintaxis

```
enumerate(iterable, start=0)
```

En el ejemplo anterior, podría interesarnos además de desempaquetar las listas, obtener los índices de por donde vamos.

Un ejemplo completo seria este:

```
In [ ]: #Ejemplo sencillo de enumerate en  
betis = ["Isco", "LoCelso", "Anthony"]  
  
for indice, jugador in enumerate(betis):  
    print(f"Índice: {indice}, jugador: {jugador}")
```

```
In [ ]: # Y para el ejemplo anterior:  
nombres = ["Isco", "LoCelso", "Anthony"]  
edades = [33, 29, 25]  
posiciones = ["MedioCentro", "MedioCentro", "Extremo"]
```

```
for i, (nombre, edad, posicion) in enumerate(zip(nombres, edades, posiciones):
    print(f"Iteración {i}: {nombre} tiene {edad} años juega de {posicion}")
```

```
In [ ]: # Diccionario vacío
jugadores = {}

# Añadir jugadores
jugadores["lamine"] = {
    "equipo": "FC Barcelona",
    "posición": "Delantero",
    "nombre": "Lamine Yamal",
    "número": 10
}
jugadores["nico"] = {
    "equipo": "Athletic Club",
    "posición": "Extremo",
    "nombre": "Nico Williams",
    "número": 10
}
jugadores["isco"] = {
    "equipo": "Real Betis",
    "posición": "Mediocentro",
    "nombre": "Don Francisco Alarcón",
    "número": 22
}

jugadores["mbappe"] = {
    "equipo": "Real Madrid",
    "posición": "Delantero",
    "nombre": "Kylian Mbappé",
    "número": 10
}

jugadores["griezmann"] = {
    "equipo": "Atlético de Madrid",
    "posición": "Delantero",
    "nombre": "Antoine Griezmann",
    "número": 7
}

jugadores["courtois"] = {
    "equipo": "Real Madrid",
    "posición": "Portero",
    "nombre": "Thibaut Courtois",
    "número": 1
}

jugadores["bartra"] = {
    "equipo": "Real Betis",
    "posición": "Defensa central",
    "nombre": "Marc Bartra",
    "número": 5
}
```

```
In [ ]: # Recorrer jugadores con enumerate
for i, (clave, datos) in enumerate(jugadores.items()):
    nombre = datos["nombre"]
    equipo = datos["equipo"]
    posicion = datos["posición"]
    numero = datos["número"]

    print(f"Iteración {i}: {nombre}, {posicion} del {equipo}, lleva el número {numero}")

    # Condición de parada
    if numero == 5:
        print(f"--> Se encontró a {nombre} que juega con el número {numero}.")
        break
```

Nota: Hasta aquí hemos trabajado con estructuras nativas de Python como listas y diccionarios.

Aunque son muy útiles para entender la lógica básica, cuando manejemos **grandes volúmenes de datos** o necesitemos hacer operaciones más avanzadas, lo más práctico será trabajar con **pandas** (para datos tabulares) y **numpy** (para cálculo numérico).

En el siguiente MODULO veremos cómo estas librerías simplifican mucho nuestro trabajo.

4. Bucle `while` en Python

El bucle `while` repite un bloque de instrucciones **mientras** una condición sea verdadera.

A diferencia del `for`, no recorre de manera explícita una lista o un rango, sino que depende de una **condición lógica** que se evalúa en cada iteración.

La sintaxis básica es:

```
while condición:
    # bloque de código
```

Un error **MUY HABITUAL** es no inicializar la variable que pondremos en la condición, o no actualizarla dentro del `while` lo que nos llevará a bucles infinitos

Diferencias clave entre `for` y `while`

- El bucle `for` recorre una secuencia (lista, rango, cadena, diccionario, etc.). Si la secuencia tiene al menos un elemento, **el bucle siempre se ejecuta al menos una vez**.

- El bucle `while` depende únicamente de que la condición inicial sea verdadera. Si la condición es falsa desde el principio, **el bucle puede no ejecutarse nunca**.

In []: *#Bucle básico*

```
i = 0
while i < 5:
    print(i)
    i += 1 # i = i+1
```

In []:

```
contador = 0
while contador < 10:
    contador += 1
    if contador % 2 == 0: #El resto en modulo 2 igual a cero implica numero
        continue # salta el resto del código y vuelve al inicio del bucle
    print("Número:", contador)
```

Bucle while con entrada del usuario

In []: *#En este ejemplo nos quedamos dentro del bucle hasta que no escribamos salir*

```
respuesta = "" #Inicializamos la variable de respuesta a una cadena vacia.
```

```
while respuesta != "salir":
    respuesta = input("Escribe algo (o 'salir' para terminar): ")
    #Ahora vamos a evitar que el usuario se nos ponga creativo
    respuesta = respuesta.lower() #toda entera en minusculas

    print("Has escrito:", respuesta)
```

In []: *#Añadimos un break para que el bucle mas no se ejecute si ha llegado a la it*

```
contador = 0
while True:
    contador += 1
    print("Iteración:", contador)
    if contador == 5:
        break
```

In []:

```
# Simulamos un proceso, por ejemplo, que lanzamos una pelota
# y que en cada rebote pierde un 75% de su energia
# y vamos imprimiendo la altura de cada rebote hasta que esta a menos
# de 1m del suelo
perdida = 0.75
altura = float(input("introduce altura del lanzamiento inicial"))
rebote = 0
while altura > 1:
    print(f"Rebote: {rebote} ---- Altura: {altura:.2f}m")
    rebote += 1
    altura *= perdida #Fijate esto es lo mismo que hacer altura = perdida*altura
#Esto nos permite hacer en una sola linea todas estas cosas
```

```
#nueva_altura = perdida * altura
#altura = nueva_altura
print(f'Ha tardado {rebote} rebotes en rebotar por debajo de 1m')
```

Podemos usar el while para quedarnos pidiendo una contraseña y si no la introduce bien a la tercera saltar una alarma

```
In [ ]: pwd = "CSIC!2025"
        tries=0
        entrada = "" #La inicializacion vacia

while entrada != pwd and tries < 3:
    entrada = input("Introduce la contraseña correcta: ")
    tries += 1

    if entrada == pwd:
        print("¡Bienvenido al curso!")
        break # Contraseña correcta, salimos del while

    elif tries == 3:
        print("Tu actitud es sospechosa y ha sido reportada")
        quit() # Cierra el programa

    else:
        print(f"Este es tu {tries} intento")

###RECUERDA si hay más de una
```

Contenido Avanzado: Manejo de Errores con try-except

En Python, algunas entradas de usuario o operaciones pueden provocar errores que detienen el programa.

Para controlar esto y evitar que el programa se rompa, usamos **try-except**.

Sintaxis básica

```
try:
    # Código que puede generar un error
    numero = int(input("Introduce un número: "))
    print("El doble es:", numero * 2)

except ValueError:
    # Este bloque se ejecuta si ocurre un ValueError
    print("¡Eso no es un número válido!")
```

```
In [ ]: #Vamos a forzar un error

numero = "A"
print(f"El cuadrado de {numero} es {numero**2}")
```

```
In [ ]: try:
    # Código que puede generar un error
    numero = int(input("Introduce un número: "))
    print(f"El cuadrado de {numero} es {numero**2}")

except ValueError:
    # Este bloque se ejecuta si ocurre un ValueError
    print("¡Eso no es un número válido!")
```

```
In [ ]: """
Contenido avanzado: pedir un número válido usando try-except y while.

En este ejemplo combinamos try-except con while para que el programa
siga pidiendo un número hasta que el usuario introduzca uno válido.
"""

numero_valido = False # Un bool que nos dice si hemos o no introducido un r

while not numero_valido:
    try:
        numero = int(input("Introduce un número: "))
        print(f"El cuadrado de {numero} es {numero**2}")
        numero_valido = True # Si llegamos aquí, la entrada es válida

    except ValueError:
        print("¡Eso no es un número válido! Inténtalo de nuevo.")
```

EJERCICIOS DEL MODULO

Ejercicios propuestos: Control de Flujo y Bucles en Python

Ejercicio 1: Número par o impar

Pide al usuario un número entero y muestra si es **par** o **impar**.

```
In [ ]: #tu código aqui
```

Ejercicio 2: Comparación de números

Pide al usuario dos números y muestra cuál es mayor, o si son iguales.

In []: *#tu código aqui*

Ejercicio 3: Clasificación de edades

Pide la edad de un usuario y clasifícala:

- Menor de 13 → "Niño"
- Entre 13 y 19 → "Adolescente"
- Entre 20 y 59 → "Adulto"
- 60 o más → "Adulto mayor"

In []: *#tu código aqui*

Ejercicio 4: Contar letras

Pide al usuario que escriba una frase y cuenta cuántas letras **mayúsculas** y **minúsculas** hay.

In []: *#tu código aqui*

Ejercicio 5: Factorial con `for`

Pide al usuario un número entero positivo y calcula su **factorial** usando un bucle `for`.

In []: *#tu código aqui*

Ejercicio 6: Filtrar números

Dada la lista `numeros = [3, 8, 12, 5, 7, 10]`, crea un bucle `for` que imprima solo los números **mayores que 6**.

In []: *#tu código aqui*

Ejercicio 7: Diccionario con `zip`

Pide al usuario tres nombres y sus edades correspondientes y construye un diccionario usando `zip`. Luego imprime:

"Nombre tiene X años"

In []: *#tu código aqui*

Ejercicio 8: Bucle `while` con condición

Crea un `while` que sume números que introduce el usuario hasta que la suma supere 50. Muestra la suma final.

```
In [ ]: #tu código aqui
```

Ejercicio 9: Palíndromo

Pide al usuario una palabra o frase y verifica si es un **palíndromo** (se lee igual al derecho y al revés). Ignora mayúsculas y espacios.

```
In [ ]: #tu código aqui
```

Ejercicio 10: Entrada hasta número impar

Pide números al usuario hasta que introduzca un **número impar**.

- Usa `while` y `try-except` para asegurarte de que el usuario introduce un número válido.
- Cuando se introduzca un número impar, imprime un mensaje de felicitación y termina.

```
In [ ]: #tu código aqui
```

```
In [ ]: # SOLUCIONES

# Ejercicio 1: Número par o impar
print("Ejercicio 1")
numero = int(input("Introduce un número: "))
if numero % 2 == 0:
    print(f"{numero} es par")
else:
    print(f"{numero} es impar")

# Ejercicio 2
print("Ejercicio 2")

a = float(input("Introduce el primer número: "))
b = float(input("Introduce el segundo número: "))

if a > b:
    print(f"{a} es mayor que {b}")
elif a < b:
    print(f"{b} es mayor que {a}")
else:
    print("Los números son iguales")

# Ejercicio 3
```

```
print("Ejercicio 3")

edad = int(input("Introduce tu edad: "))

if edad < 13:
    print("Niño")
elif edad <= 19:
    print("Adolescente")
elif edad <= 59:
    print("Adulto")
else:
    print("Adulto mayor")

# Ejercicio 4
print("Ejercicio 4")

#Solucion básica

frase = input("Introduce una frase: ")

# Inicializamos los contadores
mayusculas = 0
minusculas = 0

# Recorremos cada carácter de la frase
for c in frase:
    if c.isupper():      # Si es una letra mayúscula
        mayusculas += 1 # Incrementamos el contador de mayúsculas
    elif c.islower():   # Si es una letra minúscula
        minusculas += 1 # Incrementamos el contador de minúsculas

# Mostramos los resultados
print("Número de letras mayúsculas:", mayusculas)
print("Número de letras minúsculas:", minusculas)

#Solucion más avanzada

mayusculas = sum(1 for c in frase if c.isupper()) #sumo 1 por cada caracter
minusculas = sum(1 for c in frase if c.islower()) #sumo 1 por cada caracter

print(f"Mayúsculas: {mayusculas}, Minúsculas: {minusculas}")

# Ejercicio 5
print("Ejercicio 5")

n = int(input("Introduce un número entero positivo: "))
factorial = 1

for i in range(1, n + 1):
    factorial *= i

print(f"El factorial de {n} es {factorial}")

# Ejercicio 6
print("Ejercicio 6")
```

```
numeros = [3, 8, 12, 5, 7, 10]

for num in numeros:
    if num > 6:
        print(num)

# Ejercicio 7
print("Ejercicio 7")

nombres = [input("Nombre 1: "), input("Nombre 2: "), input("Nombre 3: ")]

edades = [int(input(f"Edad de {nombre}: ")) for nombre in nombres]

diccionario = dict(zip(nombres, edades))

for nombre, edad in diccionario.items():
    print(f"{nombre} tiene {edad} años")

# Ejercicio 8
print("Ejercicio 8")

suma = 0

while suma <= 50:
    numero = float(input("Introduce un número: "))
    suma += numero
    print(f"Suma actual: {suma}")

print(f"Suma final: {suma}")

# Ejercicio 9
print("Ejercicio 9")

texto = input("Introduce una palabra o frase: ").replace(" ", "").lower()
if texto == texto[::-1]:
    print("Es un palíndromo")
else:
    print("No es un palíndromo")

# Ejercicio 10
print("Ejercicio 10")

numero_valido = False

while True:
    try:
        numero = int(input("Introduce un número, no pararé de preguntar hasta que sea par: "))
        if numero % 2 != 0:
            print(f"¡Número impar {numero} recibido! Fin del bucle.")
            break
    except ValueError:
        print("Eso no es un número válido, inténtalo de nuevo.")
```

In []:



Videopodcast Resumen del Módulo (04)



Cuestionario de Auto-evaluación y Participación

¡Pon a prueba los conocimientos adquiridos en este módulo! Completa este breve cuestionario interactivo de 5 minutos en Google Forms:



[Cuestionario Módulo 04 en Google Forms](#)



Tutor Virtual & Libreta de Estudio (Google NotebookLM)

¿Quieres escuchar el resumen conversacional en audio de este módulo o resolver dudas con el tutor virtual del curso?



[Acceder al Espacio de Estudio Interactivo en NotebookLM](#)